Quarry: Technical Architecture Specification

Local Semantic Search for AI Agents

September 2026

Contents

1 Overview **3**
- 1.1 Design Principles . . . 3

2 System Architecture **3**
- 2.1 Deployment Topology . . . 3
- 2.2 Surface Summary . . . 4

3 Module Responsibilities **4**
- 3.1 Data Layer (`src/quarry/db/`) . . . 4
- 3.2 Ingestion Pipeline (`src/quarry/ingestion/`) . . . 4
- 3.3 Retrieval Seam (`src/quarry/retrieval/`) . . . 6
- 3.4 Format Processors (`src/quarry/extractors/`) . . . 6
- 3.5 Backends . . . 6
- 3.6 Capture and Redaction . . . 7
- 3.7 Fetch Safety (SSRF) . . . 7
- 3.8 Sync and Registry . . . 8
- 3.9 Always-on Watch/Index Loop (DES-045) . . . 9
- 3.10 Surfaces . . . 10
- 3.11 Support . . . 11

4 Daemon Model **12**
- 4.1 Why a Daemon . . . 12
- 4.2 Process Boundary and Loopback Authentication (v2.2) . . . 12
- 4.3 Client/Engine Boundary (I1) . . . 13
- 4.4 MCP Transport (`quarry mcp`) . . . 13
- 4.5 CLI and the Daemon . . . 13

5 MCP Tool Surface **14**

6 HTTP Server **14**
- 6.1 ASGI Application . . . 14
- 6.2 Endpoints . . . 15
- 6.3 Port File . . . 15

7 Configuration **15**
- 7.1 Settings . . . 15
- 7.2 Named Databases . . . 17
- 7.3 Chunking Tuning . . . 17

8 Search and Retrieval **17**
- 8.1 Embedding Model . . . 17
- 8.2 OCR . . . 17
- 8.3 Search Quality Tips . . . 17
- 8.4 Hybrid Search . . . 18
- 8.5 Retrieval Seam and Evaluation . . . 18
- 8.6 Ethos Extension Integration . . . 19

8.7 Agent-Memory Write Loop (DES-055) . . . 19
8.8 Recall Telemetry (DES-056) . . . 20
8.9 Skills and Cross-Harness Distribution (quarry-x7ja) . . . 20

9 Logging and Exceptions **21**
9.1 Logging Standard . . . 21
9.2 Exception Handling . . . 22

10 Security **22**
10.1 Bearer Token Authentication . . . 22
10.2 Request Admission (CSRF) . . . 22
10.3 CORS . . . 23
10.4 TLS and TOFU Certificate Pinning . . . 23
10.5 Log Safety (CWE-532, CWE-117) . . . 23
10.6 Capture PII Redaction (DES-036) . . . 23

11 Deployment **24**
11.1 Single Machine . . . 24
11.2 Network Mode (Remote GPU) . . . 24
11.3 Client (Remote Access) . . . 24
11.4 Daemon (`quarryd`) . . . 24
11.5 Container (Fly.io) . . . 24

12 Performance **25**
12.1 Critical Path . . . 25
12.2 Fire-and-Forget Writes . . . 25
12.3 Single Table Design . . . 25

13 Plugin and Hook System **25**
13.1 Session Start . . . 26
13.2 Web Fetch Capture . . . 26
13.3 Web Search and Read Capture . . . 26
13.4 Pre-Compact and Session-End Capture . . . 26
13.5 Subagent Stop Capture . . . 27

14 CLI Commands **27**

15 Test Architecture **28**

16 Formal Model **30**

A Operation Concurrency Model **30**
A.1 Concurrency Matrix . . . 30
A.2 Design Rationale . . . 31
A.3 Task Lifecycle . . . 31

1 Overview

Quarry is a local semantic search engine. It ingests documents in 20+ formats, embeds them with a local ONNX model, stores vectors in LanceDB, and serves semantic search to AI agents (Claude Code, Claude Desktop) and a macOS menu bar app.

The system is **daemon-first** (DES-031 v2.2, ACCEPTED and complete): a single `quarryd` process holds the engine, and the CLI, MCP, and library are pure clients that reach it over a versioned wire protocol — the daemon is the intermediary service layer. The engine runs only in `quarryd` (the `quarry serve` subcommand is retired); loopback requests are authenticated with a mode-0600 `serve.token` resolved through `ClientConfig`; and every client surface — the CLI, the `quarry mcp` stdio server, hooks, and the library — reaches the daemon through one `QuarryClient` over the `/v1` REST contract. No client process imports or constructs the engine; an import-linter contract and a runtime engine-sabotage test enforce that boundary structurally in CI (the epic `quarry-ma6f`/`quarry-ynvs` is closed). The always-on watch/index loop that daemon-first exists to enable (DES-045) keeps every registered directory's index fresh with no human action.

1.1 Design Principles

P1. **Daemon-first.** The `quarryd` process is the single engine (embedding model, LanceDB, pipeline, sync registry); the CLI, MCP, and library are clients over a versioned REST contract (DES-031 v2.2, ACCEPTED and complete). The prior *library-first* principle — the core library did all work and the surfaces were thin wrappers — described the as-built state DES-031 v2 replaced.

P2. **Local by default.** Everything runs on the user's machine. No API keys, no cloud accounts. The embedding model downloads once (∼120 MB, int8 quantized).

P3. **One daemon, many clients.** A single `quarryd` process loads the embedding model once and serves all sessions over the REST wire; supervised by launchd `KeepAlive` / systemd `Restart=always`.

P4. **Fire-and-forget I/O.** Side-effect MCP tools return immediately and process in a bounded thread pool. Search is synchronous.

P5. **Named databases.** Each database is fully isolated: its own LanceDB directory, sync registry, and vector index. Work/personal separation without configuration files.

2 System Architecture

2.1 Deployment Topology

Single machine (localhost TLS):

```
                    stdio HTTPS /v1 (TLS)
Claude Code <------------------> quarry mcp <--------------------> quarryd
           MCP JSON-RPC (QuarryClient) pinned CA cert (one daemon)
```

Remote server (GPU):

```
Mac (client) Linux (server)
                    stdio HTTPS /v1 (TLS)
Claude Code <------------------> quarry mcp <---------------------> quarryd
           MCP JSON-RPC (QuarryClient) pinned CA cert (GPU daemon)

CLI -----> HTTPS GET/POST /v1 -----> quarryd
          Bearer + pinned CA
```

Quarry's own Claude Code plugin runs `quarry mcp` directly (a FastMCP stdio server whose tools are `QuarryClient` calls over the daemon's `/v1` REST API); it no longer routes through `mcp-proxy` (DES-031 v2.2). Because a single `quarryd` holds the resident engine, every Claude Code tab shares one warm model rather than spawning a Python process that loads ∼200 MB of its own. Installed services use TLS with TOFU certificate pinning (DES-020) — even on localhost. (`mcp-proxy` is an MCP stdio↔daemon transport, not a REST-API client; `quarry install` still fetches it because it remains a

supported transport for other MCP clients, but it is no longer on quarry's own MCP path — the plugin runs `quarry mcp` directly.)

When a remote server is configured (DES-021), the CLI routes all data commands over HTTPS to the server, with only authentication and administration commands remaining local. `find`, `show`, `status`, `ingest`, `remember`, `delete`, `sync`, all `list` subcommands, and directory `register`/`deregister` route remotely. `use` is local by design: it sets the client's persistent default database, but the remote server is fixed to the database it was started with, so switching databases is a client-side setting rather than a remote call.

2.2 Surface Summary

Surface	Transport	Purpose
CLI	`QuarryClient` over HTTPS `/v1`	Interactive use from terminal
MCP server	stdio (`quarry mcp`, a `QuarryClient`)	Claude Code/Desktop tools
HTTP server	REST JSON `/v1` (FastAPI in `quarryd`)	CLI, MCP, menu bar app
Plugin	Hook shell commands (`quarry-hook`)	Auto-register; capture web fetch/search, opt-in reads, compaction, session end, subagent stop; memory distillation

3 Module Responsibilities

3.1 Data Layer (`src/quarry/db/`)

The database facade composes five subsystems behind a single entry point (`Database`), so callers pass one object rather than a raw LanceDB connection.

Module	File	Responsibility
Facade	`src/quarry/db/facade.py`	`Database` — single entry point composing the store, search, catalog, schema, and optimizer subsystems (DES-031).
Chunk store	`src/quarry/db/chunk_store.py`	Insert, progressive-flush, delete, and count operations on the `chunks` table.
Chunk search	`src/quarry/db/chunk_search.py`	Vector-similarity search under the cosine metric.
Chunk catalog	`src/quarry/db/chunk_catalog.py`	List documents, list collections, and fetch page text.
Optimizer	`src/quarry/db/optimizer.py`	Table compaction, FTS index rebuild, and per-collection indexing.
Schema	`src/quarry/db/schema.py`	Chunks-table schema and idempotent migration (agent-memory columns, FTS index).
Storage	`src/quarry/db/storage.py`	LanceDB connection management (discovery and size reporting removed — DES-049).
Connection	`src/quarry/db/connection.py`	Self-recycling LanceDB connection that bounds file-descriptor growth.
Chunk table	`src/quarry/db/chunk_table.py`	Chunks-table lifecycle: open, migrate, create-on-demand.

Supporting types: `src/quarry/models.py` (immutable `PageContent`, `Chunk`, `PageAnalysis` dataclasses), `src/quarry/types.py` (structural `Protocol`s for LanceDB, OCR, and embedding backends), `src/quarry/results.py` (`SearchResult`, `SearchFilter`, `WORST_CASE_DISTANCE`), and `src/quarry/config.py` (`Settings`, `resolve_db_paths()`, ONNX model constants).

3.2 Ingestion Pipeline (`src/quarry/ingestion/`)

Module	File
Pipeline	src/quarry/ingestion/pipeline.py
Format strategies	src/quarry/ingestion/format_strategies.py
Funnel	src/quarry/ingestion/chunk_store_funnel.py
Chunker	src/quarry/ingestion/page_chunker.py
Binary guard	src/quarry/content_guard.py
Row budget	src/quarry/db/row_budget.py
Web ingest	src/quarry/ingestion/web_ingest.py
Bulk / sitemap	src/quarry/ingestion/bulk_ingest.py, src/quarry/ingestion/sitemap_ingest.py
Web fetch	src/quarry/ingestion/web_fetch.py, src/quarry/ingestion/fetched_body.py, src/quarry/i
Ingest stats	src/quarry/ingestion/ingest_stats.py

Format extraction lives in src/quarry/extractors/; embedding and OCR provider selection in

src/quarry/ingestion/provider.py, src/quarry/ingestion/backends.py, and src/quarry/ingestion/ocr_local.

3.3 Retrieval Seam (src/quarry/retrieval/)

Hybrid search runs through one parameterizable seam shared by every surface and by the eval harness, so the production path and the committed baseline cannot drift.

Module	File	Responsibility
Service	src/quarry/retrieval/service.py	SearchService — the single call every surface (CLI, HTTP, MCP) makes for search. Kills remote/local divergence (bug class 3).
Hybrid	src/quarry/retrieval/hybrid.py	HybridRetriever — vector + BM25 channels fused with RRF, behind a config.
Config	src/quarry/retrieval/config.py	RetrievalConfig — frozen retrieval knobs (RRF k, over-fetch, cosine metric, decay rate, reranker). Defaults reproduce shipped search bit-for-bit.
Protocols	src/quarry/retrieval/protocols.py	Retriever and Reranker structural interfaces (one method each).
Fusion	src/quarry/retrieval/fusion.py	RrfFusion — Reciprocal Rank Fusion with optional temporal decay.
Reranker	src/quarry/retrieval/reranker.py	NullReranker — pass-through Null Object when reranking is off.

3.4 Format Processors (src/quarry/extractors/)

Each extractor implements a common Protocol (src/quarry/extractors/protocol.py) and converts a source format into list[PageContent]:

Module	File	Formats
PDF	src/quarry/extractors/pdf_extractor.py	PDF (text via PyMuPDF, OCR fallback for image pages)
Image	src/quarry/extractors/image_extractor.py	PNG, JPG, TIFF, BMP, WebP (OCR)
Text	src/quarry/extractors/text_extractor.py	TXT, MD, LaTeX, DOCX
Code	src/quarry/extractors/code_extractor.py	30+ languages (tree-sitter section splitting)
HTML	src/quarry/extractors/html_extractor.py	HTML (boilerplate stripping, Markdown conversion)
Spreadsheet	src/quarry/extractors/spreadsheet_extractor.py	XLSX, CSV (LaTeX tabular serialization)
Presentation	src/quarry/extractors/presentation_extractor.py	PPTX (slide-per-chunk with notes)

3.5 Backends

Module	File	Responsibility
Factory	src/quarry/ingestion/backends.py	Thread-safe factory with double-checked locking. get_ocr_backend() and get_embedding_backend() return cached singletons.
OCR	src/quarry/ingestion/ocr_local.py	Local OCR via RapidOCR (offline, no setup).
Embeddings	src/quarry/embeddings.py	Local ONNX embedding (snowflake-arctic-embed-m-v1.5, 768-dim, 512 tokens). Vectors are L2-normalized to unit length.
Provider	src/quarry/ingestion/provider.py	ONNX Runtime execution provider auto-detection (CUDA/FP16 or CPU/int8).

3.6 Capture and Redaction

Every automatic capture — session transcripts (PreCompact, SessionEnd, SubagentStop), WebFetch pages, WebSearch digests, and opt-in Read captures — is scrubbed at write time before any bytes reach a git-tracked (and pushable) surface (DES-036), and again server-side by the daemon's `ScrubbedIngestJob` before a chunk is stored (DES-041).

Module	File	Responsibility
Capture writer	`src/quarry/capture.py`	`CaptureWriter` — the s ery local `.md` capture pro Scrubs, then atomically
Scrubber	`src/quarry/scrub.py`	`Scrubber` — ordered red paths, emails, hostname tent.
Scrub rules	`src/quarry/scrub_rules.py`	Secret-detection regex c scrubber.
Capture URL	`src/quarry/capture_url.py`	`CaptureUrl` — tolerant that `HttpUrl` refuses; `HttpUrl.shown` and `Dis`
Web capture	`src/quarry/web_capture.py`	Parse a PostToolUse W URL + fetched content.
Transcript	`src/quarry/transcript.py`	The session transcript fil tifiers.
Session capture	`src/quarry/session_transcript.py`	`SessionTranscriptCap` archive the raw `~/.punt-labs/quarry/` write the scrubbed `CaptureIngestRequest` Compact, SessionEnd, a
Subagent capture	`src/quarry/subagent_capture.py`, `src/quarry/subagent_report.py`	`SubagentCapture` — th plus the subagent's fi an `observation` in `mem` 055).
Search / read	`src/quarry/web_search_capture.py`, `src/quarry/read_capture.py`	Parse a WebSearch res check admission filter for ture.
Daemon sender	`src/quarry/daemon_capture.py`	`DaemonCaptureSender` bounded door to `/v1/c` and `/v1/remember`.
Backfill	`src/quarry/backfill.py`	Backfill historical sessio the capture writer.

3.7 Fetch Safety (SSRF)

Every outbound fetch — the leaf-page fetch and the sitemap crawl — runs the address-safety gate (`UrlSafetyCheck`) against the *resolved* address on *every* hop, fail-closed, before the socket opens (DES-043). The route boundary validates only the initial caller-supplied URL; the gap this closes is that redirect targets and sitemap-discovered URLs never reached that gate — an authenticated caller could otherwise reach `169.254.169.254`, loopback, or a private range via a redirect from a public server or an internal URL listed in a sitemap.

Module	File	Responsibility
URL safety	`src/quarry/url_safety.py`	`UrlSafetyCheck` — classifies a URL's resolved address (rejects link-local, loopback, RFC-1918, CGNAT, unspecified, IPv4-mapped-IPv6, and the NAT64 well-known prefix; checks all `getaddrinfo` records). In core, so the daemon route and the fetch layer share one classifier.
Redirect guard	`src/quarry/ingestion/ssrf_redirect.py`	`SsrfGuardedRedirectHandler` plus the shared `GUARDED_OPENER`: re-gates every 30x `Location` before it is opened.
Sitemap client	`src/quarry/sitemap_web_client.py`	`GatedSitemapWebClient` — USP's `AbstractWebClient`, gating every sitemap-index, `robots.txt`, and sub-sitemap fetch at every recursion depth (a block is non-retryable, so USP never connects).
Pinned connection	`src/quarry/ingestion/pinned_connection.py`	`PinnedHTTP(S)Connection` — performs one `getaddrinfo`, validates every record, and connects the socket to a validated IP literal from that same set; no second resolution exists, so the gate-vs-connect DNS-rebind window is eliminated by construction (DES-044). Installed into the shared `GUARDED_OPENER`, so every fetch path and redirect hop inherits the pin.

A single `getaddrinfo` per connection eliminates the gate-vs-socket DNS-rebind TOCTOU by construction: the socket connects to an address the gate has just validated, never an independent re-resolution (DES-044, closing quarry-kmzo/quarry-ljym). The pin narrows the *address*, not the *trust* — `self.host` is never mutated, so SNI, the `Host` header, and certificate verification stay on the hostname (never the pinned IP), and the public-fetch context keeps the system trust store, kept separate from the daemon-RPC pinned-CA context.

3.8 Sync and Registry

Module	File	Responsibility
Sync	`src/quarry/sync.py`	Directory sync: discover files, compute delta (new/changed/deleted), ingest/delete accordingly.
Discovery	`src/quarry/sync_discovery.py`	Walk, filter, and hash files for the sync delta.
Ingest	`src/quarry/sync_ingest.py`	Progressive per-collection ingest (producer/-consumer with within-file resume).
Registry	`src/quarry/sync_registry.py`	SQLite registry of watched directories and per-file state.
Planner	`src/quarry/sync_planner.py`	Compute the disk-vs-registry plan for one collection (TOCTOU-resilient; the one re-embed rule lives in `src/quarry/sync_change.py`).
Lock	`src/quarry/sync_lock.py`	`SyncLock` — the kernel-`flock` single-flight guard around the detached background sync launch.
Finalize	`src/quarry/sync_finalize.py`	Post-sync index rebuild, optimize, capture-shadow push, and GC.
Resume	`src/quarry/sync_resume.py`	Within-file resume policy: watermark gate and partial-hash marks (DES-034).

3.9 Always-on Watch/Index Loop (DES-045)

Continuous incremental indexing — the first-class daemon-first rationale (DES-031) — is a daemon-owned `src/quarry/daemon/watch_loop.py` `WatchLoop` that keeps every registered directory's index fresh with no human action. It is a *producer only*: it watches the filesystem, debounces edit bursts, and submits index work to the existing DES-042 queue, writing no LanceDB table directly. So it inherits the whole serialization stack unchanged — DES-042 per-table FIFO → DES-034 `ProgressiveIndexer`/`_write_lock` → DES-026 WAL — and the queue's routing key becomes `(database, collection)`, extending single-writer-per-table across the roster. Three producers (initial scan on start, the live watch, an explicit `quarry sync`) all feed the one queue and thus cannot race.

Module	File	Responsibility
Watch loop	`src/quarry/daemon/watch_loop.py`	The `WatchLoop`: per-database watch sets + observer lifecycle, initial scan, start/stop watching, `request_scan`. Producer into the queue.
Reconcile	`src/quarry/daemon/watch_reconcile.py`	The `watch_safety_scan_s` roster reconcile: re-enumerate, start watching new databases, re-submit shed scans, run the orphan sweep (DES-045a).
Submit	`src/quarry/daemon/watch_submit.py`	Turns a debounced delta into per-file or bulk `IngestUnit`s and enqueues them.
Roster	`src/quarry/daemon/watch_roster.py`	Per-database connections + watch sets enumerated from the on-disk roster (trust boundary: only roster DBs are opened).
Scratch guard	`src/quarry/scratch_paths.py`	`ScratchGuard`: refuses a temp/scratch/VCS root and skips gitignored paths at every producer seam (DES-045b).
Removal	`src/quarry/daemon/registration_lifecycle.py`	Deregister / subsume / retain: marks a collection in `pending_purge_collections` on non-keep-data removal; records retained-collection identity (DES-045a).
Purge	`src/quarry/daemon/purge_service.py`	The orphan sweep + `CollectionPurgeJob`: deletes only marked collections still holding chunks, re-checked under its own transaction.
Route key	`src/quarry/daemon/route_key.py`	`RouteKey = (database, collection)` — the queue routing key that extends single-writer-per-table across the roster.
Events	`src/quarry/daemon/fs_events.py`	`FsEvent` + `FsEventSource` protocol seam (tests inject synthetic events).
Watchdog	`src/quarry/daemon/fs_watchdog.py`	The sole `watchdog` importer: FSEvents (macOS) / inotify (Linux) with a `PollingObserver` fallback (on `ENOSPC`).
Debounce	`src/quarry/daemon/debounce.py`	Per-path debounce window + max-delay cap + bulk-threshold coalescing.
Index jobs	`src/quarry/daemon/index_jobs.py`	`FileIndexJob` / `DocumentDeleteJob` / `CollectionSyncJob` (`IngestUnit`s).
Finalize	`src/quarry/daemon/finalize_job.py`	`CollectionFinalizeJob` (`SyncFinalizer`): the lone post-quiescence `optimize()` + FTS rebuild, coalesced by `FinalizeThrottle` (DES-045c).

The loop feeds the same DES-042 serialized queue (`src/quarry/daemon/ingest_queue.py`, `src/quarry/daemon/coll` one resident FIFO worker per `RouteKey` (one in-flight `progressive_insert` per LanceDB table) behind a global embed `Semaphore` hard-clamped to 1 (DES-032: more than one buys no matmul parallelism behind the shared ONNX session mutex and re-adds arena contention). Admission is non-blocking and bounded — a full queue returns 503 and the event re-arms with capped backoff, never a silent drop.

A small delta re-indexes as per-file jobs (embed gate released between files for cross-collection fairness); a burst above `watch_bulk_threshold` (default 50) collapses to one `CollectionSyncJob` (DES-034's size-gated flush keeps the fragment budget). Per-file jobs *never* rebuild the FTS index — a lone finalize job runs post-quiescence per `RouteKey`, so the daemon's persistent connection does not reopen the quarry-0dss descriptor leak (`create_fts_index(replace=True)` pins deleted-file readers the LanceDB Rust core never evicts); a resource-invariant test proves the fd count plateaus across multiple databases.

Removal lifecycle and orphan sweep (DES-045a). Deregister, subsumption, and orphan cleanup were hardened under a Z model (`docs/spec/watch_lifecycle.tex`, fuzz-clean and ProB-checked against ten invariants I1–I10). A collection is marked in a durable `pending_purge_collections` table — in the *same* transaction as the `directories` delete — only on a non-keep-data deregister or a subsume eviction; the periodic reconcile's orphan sweep then purges only *marked* collections that still hold chunks, re-checking under its own transaction at execution time. Non-directory collections (captures, `memory-*`, the `default remember` target, URL-host collections) are never marked, so the sweep is *structurally unable* to delete them (I9) — the invariant whose earlier absence, in a model that wrongly assumed every collection was a directory registration, would have wiped all captures and agent memories on the 5-minute default timer. A `--keep-data` disable archives the collection's identity (`retained_collections` with `original_directory`); only the *same* directory may re-adopt it (I7), and re-adoption triggers a reconcile that prunes chunks for files deleted while it was disabled. A different directory always gets a fresh, distinct collection, and the fresh-name picker avoids every chunk-bearing name so the auto path is merge-proof (I10).

Watch scope (DES-045b). The `ScratchGuard` predicate, consulted at every producer seam (initial scan, live watch, reconcile, explicit `quarry sync`), refuses a root that is (a) an OS temp root (`/tmp`, `/private/tmp`, `/var/tmp`, `/private/var/tmp`, `/var/folders`, compared casefolded so a case variant on APFS cannot slip past) or (b) a repo's own scratch, anchored on `<gitroot>/.tmp` for *any* ancestor git repo; below a permitted root it skips gitignored paths and an always-skip set (`.git`, `.beads`, `.venv`, `node_modules`, `__pycache__`, `dist`, `build`, caches). Resolution is fail-closed *per root* — a `resolve()` that raises skips that one registration and logs, never aborting the scan. The guard fires at *watch* time, not only at register time, so an already-registered temp root is refused on daemon restart with no registry edit. This closes the hole where a stale `/private/tmp` registration made the loop re-index the churning system temp dir and never reach quiescence (283 CPU-minutes observed before the guard shipped).

CPU/compaction envelope (DES-045c). DES-045's "bounded by construction" bounded fds, memory, and the write-serializer, but LanceDB sizes its own tokio *compute* runtime to `num_cpus`, and a single finalize (`optimize()` + native FTS rebuild) on that uncapped pool seized every core (291–403 % CPU observed). Two structural mitigations bound it: (1) `ThreadConfig.apply_env_limits()` (`src/quarry/thread_config.py`) clamps `LANCE_CPU_THREADS`/`LANCE_IO_THREADS` (cap 2) before the first `lancedb.connect` in `DaemonServer.run()`, via a fail-closed clamp with a per-variable floor (a stale `LANCE_CPU_THREADS=32` is forced down to the cap, and `=1` — which stalls lance's runtime — is raised to the floor of 2, so floor equals ceiling for the LANCE pools); (2) `FinalizeThrottle` coalesces the finalize to at most one per `watch_optimize_min_interval_s` (default 30 s) *per database*, with a trailing timer so FTS still catches up after churn settles. The vector channel is never stale (files index immediately); the reconcile is the unthrottled backstop.

3.10 Surfaces

Module	File	Responsibility
CLI	src/quarry/__main__.py + src/quarry/cli_*.py	Typer CLI with rich formatting. A QuarryClient client: build reques call daemon → format output. Com groups split into src/quarry/cli_se src/quarry/cli_ingest, src/quarry/cli_documents, src/quarry/cli_sync, src/quarry/cli_project, src/quarry/cli_remote, src/quarry/cli_maintenance, src/quarry/cli_captures, src/quarry/cli_missions; shared o plumbing in src/quarry/cli_formatte
MCP Server	src/quarry/mcp_server.py, src/quarry/mcp_missions.py	quarry mcp: a FastMCP *stdio* server v tool bodies (the search/ingest/manage tools on McpTools, plus missions on the sibling MissionTools, all b src/quarry/mcp_guard.py's ToolGuard thin QuarryClient calls over the daer /v1 API. Imports *no* engine — loads LanceDB/ONNX (DES-031 v2.2).
Client tier	src/quarry/client/	QuarryClient + injectable REST trans typed QuarryError hierarchy, ClientCo target/token resolution. Replaces the de RemoteClient.
Wire API	src/quarry/api/	The /v1 contract: one Pydantic reques sponse model per operation + a un ErrorBody. Drift is an import-time type (the structural kill of bug class 3).
Hooks	src/quarry/hooks.py, src/quarry/hooks_agent.py	Claude Code plugin hooks: session- (auto-register), post-web-fetch, pre-con (src/quarry/hooks.py); session post-web-search, post-read, subagent (src/quarry/hooks_agent.py). Pure c (QuarryClient); all fail-open.
Hook Entry	src/quarry/_hook_entry.py	Lightweight hook dispatcher that bypasse CLI import chain. Dispatches via sys. with lazy imports (∼0.1 s vs ∼1.5 s fo CLI).
Remote	src/quarry/remote.py	Remote login config: TOFU CA cert fetc storage, pinned-CA connection validatio
Daemon	src/quarry/service.py	quarryd lifecycle via launchd/systemd.

3.11 Support

Module	File	Responsibility
Formatting	src/quarry/formatting.py	Pre-formatted plain text outpu box-drawing. Used by both CL consistent output.
Sitemap	src/quarry/sitemap.py	Sitemap discovery and URL web ingestion.
Doctor	src/quarry/doctor.py + src/quarry/doctor_*.py	Health checks and install flow ability, database access, MCP daemon, sync, captures, ethos telemetry (src/quarry/doct src/quarry/doctor_ethos.py src/quarry/doctor_recall.p
Recall Telemetry	src/quarry/query_log.py + src/quarry/query_log_schema.py	Local, engine-free SQLite st telemetry (DES-056): one que + N query_hits rows per searc src/quarry/daemon/routes/s read back through GET /insi src/quarry/sync_registry.p lifecycle.
Enable	src/quarry/enable.py	Project activation and register directory, refresh dored ethos memory-g (src/quarry/ethos_ext_bloc src/quarry/ethos_ext_scan. per-project config.
TLS	src/quarry/tls.py	TLS certificate generation: se 256 CA and server cert with at and TOFU-safe CA reuse.
Proxy	src/quarry/proxy.py	mcp-proxy binary download, cation, and platform-specific in
Logging	src/quarry/logging_config.py	Structured logging configuratio handler + stderr, QUARRY_LOG_

4 Daemon Model

4.1 Why a Daemon

The embedding model (snowflake-arctic-embed-m-v1.5, int8 quantized ONNX) occupies ∼200 MB of RAM and takes ∼250 ms to load. MCP's stdio transport model spawns a new process per session. Without a daemon, five Claude Code tabs means five copies of the model in memory and five cold starts.

`quarryd` loads the model once and keeps it resident, respawned by launchd `KeepAlive` / systemd `Restart=always`.

4.2 Process Boundary and Loopback Authentication (v2.2)

`quarryd` (`src/quarry/daemon/launcher.py`) is a dedicated entry point — the sole engine process, making the client/engine boundary a hard *process* split. Only `src/quarry/daemon/` imports the engine; the `quarry serve` subcommand is deleted.

At startup `quarryd` refuses a remote-reachable bind that carries no operator key (an auto-generated token would be false security — it is unreadable by the remote clients that need it), and for a loopback bind mints a 256-bit `serve.token` when none is supplied. The daemon writes that token mode-0600 beside `serve.port` (shared `src/quarry/run_dir.py`) via an atomic `os.open(0o600)` + tmp-rename, and requires it on every request — closing the multi-user-host exposure where any local UID could reach the unauthenticated daemon on `127.0.0.1`. Loopback classification (`src/quarry/net.py`, `LoopbackPolicy`) is `ipaddress`-based: `localhost`/`::1`/`127.0.0.0/8` are loopback, `0.0.0.0` and unresolved names fail closed.

Clients resolve the target through `ClientConfig` (`src/quarry/client/config.py`): for a loopback URL the bearer is read *live* from `serve.token` (it rotates every daemon restart, so a stored token is

never trusted); for a remote URL the stored bearer is kept. A missing loopback token fails closed with an actionable error rather than a bare 401. `ClientConfig` feeds the one `QuarryClient` that the CLI, the `quarry mcp` stdio server, hooks, and the library all use; `RemoteClient` is deleted.

The daemon serves the **REST API only** under `/v1` (the in-daemon MCP route was removed in PR-1); its request-admission trust model combines the `JsonContentTypeGuard` — an ASGI CSRF choke point that rejects any `POST` without `Content-Type: application/json` (a browser cannot send that cross-origin without a preflight the CORS policy refuses) — with the `serve.token` bearer above. The guard blocks a forged cross-origin write; the token blocks an unauthorized local reader.

4.3 Client/Engine Boundary (I1)

The daemon-first split (DES-031) is a hard boundary, enforced *structurally* rather than by convention: a client process never imports the engine. Three layers, dependency arrow pointing inward: `src/quarry/quarry/api` (the wire contract — Pydantic models, zero engine, zero heavy deps) ← `src/quarry/quarry/client` (`QuarryClient` transport + typed errors, imports only `api`) ← the client processes (`src/quarry/__main__`, `src/quarry/hooks`, `src/quarry/mcp_server`). The engine — `src/quarry/db/`, `src/quarry/embeddings`, `src/quarry/ingestion/`, `src/quarry/retrieval/`, `src/quarry/sync`, `src/quarry/daemon/` — is server-side only; its entry point `src/quarry/daemon/launcher.py` (the `quarryd` script) lives inside `src/quarry/daemon/`, so it is engine-side and never a client.

Two mechanisms lock it (PR-6):

- **An import-linter contract** (`.importlinter`, run by `make check-imports` in `make check` and CI) forbids any of the five client modules from importing any of the six engine packages along a *transitive* chain; a violating import fails CI, not review. The one sanctioned exception is an explicit, enumerated `ignore_imports` list of the host-admin diagnostic edges (`doctor/install/uninstall` probe the local engine environment to report why a daemon is unhealthy) — function-body lazy imports off the hot path, not an invisibility trick (grimp resolves lazy imports).
- **A runtime engine-sabotage test** imports each client module in a subprocess with `lancedb`/`onnxruntime`/`pyarrow` poisoned and asserts no engine module reaches *module* scope — the companion guard that the diagnostic lazy imports stay deferred.

Daemon-dependent tests run against a hermetic in-process ASGI fixture (`httpx.ASGITransport` over the FastAPI `build_app`, no socket); a single real-loopback-TLS smoke (slow tier, excluded from the fast suite) covers the pinned-CA wire contract end-to-end; and `install` gates on `/health ready` before exiting 0.

4.4 MCP Transport (`quarry mcp`)

Quarry's plugin runs `quarry mcp` directly — a FastMCP *stdio* server (`src/quarry/mcp_server.py`) whose tool bodies are thin `QuarryClient` calls over the daemon's `/v1` REST API. It reads MCP JSON-RPC from stdin, dispatches each tool to the resident daemon (reusing its one warm `ctx.embedder`/`ctx.database`), and relays the response to stdout. It imports no engine, so `import quarry.mcp_server` and running `quarry mcp` load zero LanceDB/ONNX. Database selection is a per-request parameter carried in the wire call; the daemon holds no per-connection `ContextVar` because the in-daemon `/mcp` WebSocket route was removed (DES-031 v2.2, PR-1). `mcp-proxy` is no longer on this path (it remains a supported MCP stdio↔daemon transport for other MCP clients).

4.5 CLI and the Daemon

The CLI is a thin client: it builds a request, calls the daemon over the `/v1` wire protocol via `QuarryClient`, and renders the response — it does not load the embedding model or open LanceDB in-process. The prior design had `quarry find "margins"` load the model in-process and query LanceDB directly; that local-engine path is removed. The one sanctioned exception is the host-admin diagnostics (`doctor`/`install`/`uninstall`), which probe the local engine environment through function-body lazy imports so they can report *why* a daemon is unhealthy even when it is down — an explicit, enumerated `ignore_imports` list, proven off the hot path by the sabotage test.

5 MCP Tool Surface

Tool	Key Parameters	Purpose	Exec
`find`	`query`, `limit`, `collection`, `agent_handle`, `memory_type`	Hybrid search	Sync
`ingest`	`source`, `overwrite`, `collection`	Ingest a URL (files go through `register_directory`)	Bg
`remember`	`content`, `document_name`, `agent_handle`, `memory_type`	Ingest inline text	Bg
`learn`	`lesson`, `topic`, `name`	Save a distilled lesson (`memory_type = lesson`)	Bg
`list`	`kind`	List indexed data	Sync
`show`	`document_name`, `page_number`	Document metadata or page	Sync
`delete`	`name`, `kind`	Delete document or collection	Bg
`register_directory`	`directory`, `collection`	Register for sync	Bg
`deregister_directory`	`collection`, `keep_data`	Remove registration	Bg
`sync_all_registrations`	—	Sync all registered dirs	Bg
`status`	—	Database stats	Sync
`use`	`name`	Switch active database	Sync
`missions_sync`	`mission`, `dry_run`, `force`	File frozen ethos mission rounds into `memory-<worker>`	Sync*

Bg = background (daemon-side; the tool returns a `task_id`). *`missions_sync` runs client-side: it reads the repo's `.punt-labs/ethos/missions/` tree and issues one `/v1/remember` (202) per frozen round the daemon does not yet hold, returning after the last post (DES-055). The table above is the registered surface: every tool on `McpTools` plus `missions_sync` on the sibling `MissionTools` (`src/quarry/mcp_missions.py`), registered from one call; the exact set is pinned by `tests/test_mcp_server.py`.

Each tool body is a thin `QuarryClient` call over `/v1` (the MCP server holds no engine). A background tool posts to the daemon, which returns `202 Accepted` with a `task_id`; the actual work runs on the daemon's serialized per-`RouteKey` queue (DES-042) behind the embed `Semaphore`, and the caller polls `/v1/tasks/{task_id}` or ignores the outcome. This keeps a 30-second ingest from blocking the LLM response stream while preventing the concurrent-write races the single-writer-per-table invariant closes. Read tools return their result inline.

6 HTTP Server

6.1 ASGI Application

FastAPI app built by `build_app()` (`src/quarry/daemon/app.py`), served by uvicorn. FastAPI *is* Starlette underneath, so request validation *and* the OpenAPI document derive from the same `src/quarry/quarry/api` Pydantic models — no hand-rolled emitter or route registry. REST handlers are sync functions run in the threadpool; the routes live in `src/quarry/daemon/routes/`. There is no WebSocket endpoint — the daemon serves the **REST API only** (the in-daemon `/mcp` route was removed in DES-031 v2.2, PR-1).

Default port: 8420 (`DEFAULT_PORT` in `src/quarry/config.py`), overridable with `--port`. All engine operations sit under `/v1`; `/health` and `/ca.crt` are unversioned. `/health` advertises `api_version`; a major mismatch raises `QuarryVersionError` in the client.

6.2 Endpoints

Path	Method	Purpose	Auth	Model
/health	GET	Server state + api_version	No	sync
/ca.crt	GET	CA certificate PEM (TOFU)	No	sync
/v1/search	GET	Hybrid search	Yes	sync
/v1/show	GET	Document metadata or page text	Yes	sync
/v1/documents	GET	List documents	Yes	sync
/v1/documents	DELETE	Delete document (returns 202)	Yes	async
/v1/collections	GET	List collections	Yes	sync
/v1/collections	DELETE	Delete collection (returns 202)	Yes	async
/v1/registrations	GET	List directory registrations	Yes	sync
/v1/registrations	POST	Register directory (returns 202)	Yes	async
/v1/registrations	DELETE	Deregister directory (sync + 202 purge)	Yes	async
/v1/databases	GET	List named databases	Yes	sync
/v1/status	GET	Database statistics	Yes	sync
/v1/insights	GET	Recall telemetry snapshot (DES-056)	Yes	sync
/v1/coverage	GET	Per-collection coverage counts	Yes	sync
/v1/use	POST	Switch active database	Yes	sync
/v1/remember	POST	Ingest inline content (returns 202)	Yes	async
/v1/learn	POST	Save a distilled lesson (returns 202)	Yes	async
/v1/ingest	POST	Ingest a URL (returns 202)	Yes	async
/v1/capture	POST	Capture web/transcript (returns 202)	Yes	async
/v1/captures/lookup	POST	Prior capture of a URL (web-fetch loop closer)	Yes	sync
/v1/captures/push	POST	Push private capture shadow repo	Yes	async
/v1/sync	POST	Sync registrations (returns 202)	Yes	async
/v1/optimize	POST	Compact + rebuild index (returns 202)	Yes	async
/v1/backfill-sessions	POST	Backfill session transcripts (returns 202)	Yes	async
/v1/tasks/{task_id}	GET	Poll task status (unified)	Yes	sync

/v1/tasks/{task_id} is the single, unified poll endpoint — the old /sync/{id} and /ingest/{id} alias routes were removed in the DES-031 v2.2 contract rewrite. See Appendix A for the full concurrency matrix across HTTP, CLI, and MCP surfaces.

6.3 Port File

After binding, the daemon writes the actual port to ~/.punt-labs/quarry/data/<db>/serve.port, and (when it has an effective key) a mode-0600 serve.token beside it (see §4.2). This supports --port 0 (ephemeral assignment). Removed on shutdown via the lifespan context.

7 Configuration

7.1 Settings

All settings are environment variables read via pydantic-settings.

Variable	Default	Description
Paths		
QUARRY_ROOT	~/.punt-labs/quarry/data	Base directory for all databases
LANCEDB_PATH	<root>/default/lancedb	Vector database location

Variable	Default	Description
REGISTRY_PATH	<root>/default/registry.db	Directory sync SQLite database
TELEMETRY_PATH	<root>/default/telemetry.db	Recall-telemetry SQLite database (DES
QUARRY_LOG_DIR	~/.punt-labs/quarry/logs/	Log directory (quarry.log for the CLI client tier, quarryd.log for the daem DES-048; rotating, 5 MB × 5; resolvab call via QUARRY_LOG_DIR)
Chunking		
CHUNK_MAX_CHARS	1800	Target max characters per chunk (∼4 kens)
CHUNK_OVERLAP_CHARS	200	Overlap between consecutive chunks
Embedding		
EMBEDDING_MODEL	Snowflake/snowflake-arctic-embed-m-v1.5	Model identifier (display only — the C model is fixed)
EMBEDDING_DIMENSION	768	Vector dimension (display only — fixed a
Progressive commit (DES-034)		
SYNC_FLUSH_MB	32	Flush buffered vectors to LanceDB onc reach this many MB (bounds peak re vectors; may fire mid-file)
EMBED_WINDOW_CHUNKS	512	Chunks embedded per window, so a larg ument never materializes all its vectors throughput seam)
Recall telemetry (DES-056)		
TELEMETRY_ENABLED	true	Record one query_events row + N query rows per search; false makes the write a true no-op
TELEMETRY_RETENTION_DAYS	90	Prune query-log rows older than this days
Watch/index loop (DES-045)		
WATCH_ENABLED	true	Run the always-on watch/index loop stalling quarryd is the opt-in; reverse prior timer posture)
WATCH_DEBOUNCE_S	1.0	Coalesce an edit burst before submittin
WATCH_MAX_DELAY_S	5.0	Cap on a continuously-rearmed path starvation)
WATCH_BULK_THRESHOLD	50	A delta above this many paths collapses bulk CollectionSyncJob
WATCH_USE_POLLING	false	Force watchdog's stat-walk PollingObs
WATCH_SAFETY_SCAN_S	300.0	Periodic roster reconcile + orphan s picks up new databases and re-submit scans (0 = off)
WATCH_OPTIMIZE_MIN_INTERVAL_S	30.0	FinalizeThrottle: at most one finalize mize + FTS rebuild) per interval *per da* (0 = every batch)
Thread limits (process env; DES-032/045c)		
LANCE_CPU_THREADS / LANCE_IO_THREADS	2	Clamp LanceDB's tokio compute/IO (fail-closed, floor == cap == 2; set befo first lancedb.connect)
OMP_NUM_THREADS	min(2, ncpu)	Cap OpenMP to match the ONNX in limit (floor 1)

The LanceDB and OMP caps are applied by ThreadConfig (src/quarry/thread_config.py) in DaemonServer.run() via a fail-closed clamp: an inherited or operator value is honored only within [floor, cap]; a higher or non-numeric value is forced *down* to the cap (a setdefault would yield upward to a stale export and void the ceiling) and a below-floor value is raised to the floor. ONNX

`intra_op_num_threads` is `min(2, ncpu)` on CPU and 1 on GPU; `inter_op_num_threads` is 1 (DES-032).

7.2 Named Databases

`resolve_db_paths(settings, name)` maps a database name to a path under `QUARRY_ROOT`. Each database is fully isolated — its own LanceDB directory, sync registry, and vector index. Names containing path separators (`/` or `\`) or the literal traversal segments `.` or `..` are rejected; names like `foo.bar` are allowed. If `LANCEDB_PATH` was explicitly overridden, the override is preserved.

The persistent default database is stored in `~/.punt-labs/quarry/config.toml` as `[default] database = "name"`.

7.3 Chunking Tuning

Increase `CHUNK_MAX_CHARS` (3000–4000) for documents with long self-contained sections: legal contracts, academic papers, large source code functions.

Decrease `CHUNK_MAX_CHARS` (800–1200) for dense documents where each paragraph covers a distinct topic.

Increase `CHUNK_OVERLAP_CHARS` (400) when relevant information spans paragraph boundaries.

The embedding model has a 512-token context window. Chunks exceeding this are truncated during embedding, reducing quality for the tail.

8 Search and Retrieval

8.1 Embedding Model

snowflake-arctic-embed-m-v1.5 via ONNX Runtime:

- 768-dimensional vectors, 512-token context window
- Asymmetric retrieval: queries are prefixed with a search instruction, documents are not
- Output vectors are L2-normalized to unit length, so search runs under the **cosine** metric and similarity ($1 - \text{distance}$) is bounded true cosine in $[-1, 1]$ (DES-038)
- int8 quantized ONNX for CPU (∼120 MB), FP16 for CUDA (∼220 MB)
- Provider auto-detection: CUDA+FP16 when available, CPU+int8 fallback. `QUARRY_PROVIDER` env var overrides (`cpu`, `cuda`, or unset for auto). See DES-016 and `src/quarry/ingestion/provider.py`.
- Auto-downloads on first use via `huggingface_hub`

The model is fixed — there is no configuration to swap it. Changing the model invalidates all existing embeddings, requiring full re-ingestion.

8.2 OCR

RapidOCR handles all image-based text extraction. For semantic search, OCR accuracy matters less than expected — the embedding model is robust to minor OCR errors. A misspelled word rarely changes semantic meaning enough to affect search ranking.

8.3 Search Quality Tips

1. **Use natural language queries.** The model is trained on question-passage pairs. "What were Q3 revenue figures?" works better than "Q3 revenue."
2. **Use collection filtering.** `quarry find "auth logic" -c backend`.
3. **Re-ingest after tuning.** Chunk parameters only affect new ingestions. Run `quarry sync` after changes.
4. **Check chunk boundaries.** `quarry find -n 1 "query"` and examine the result.
5. **Use `--overwrite` when re-ingesting.** Without it, duplicate chunks accumulate.

8.4 Hybrid Search

Hybrid search is the **default for every query** — there is no vector-only path. All three surfaces (CLI, HTTP, MCP) construct a `SearchService` (`src/quarry/retrieval/service.py`) and call its one `search` method, so the surfaces cannot drift into different retrieval behavior. This closes bug class 3 (remote/local divergence), where the HTTP `/search` endpoint once ran vector-only search while the CLI ran hybrid. `SearchService` wraps a `HybridRetriever` parameterized by a `RetrievalConfig` whose defaults reproduce the shipped pipeline bit-for-bit.

Channels:

1. **Vector similarity** — ANN search via LanceDB under the cosine metric
2. **BM25 full-text search** — LanceDB's *native* (Rust) inverted index on the `text` column, the FTS backend LanceDB builds by default, created during schema migration. Quarry's `LanceTable` protocol and the `RecyclingTable` wrapper forward only `replace` to `create_fts_index`; there is no separate Tantivy `IndexWriter` process or thread pool, so the index rides the daemon's capped LanceDB runtime (DES-045c). (Earlier ADRs — DES-017, DES-023 — name the backend "Tantivy"; that name is legacy, the shipped engine is LanceDB-native.)

Results are fused using Reciprocal Rank Fusion (RRF):

$$\text{score}(m) = \sum_{\text{channel}} \frac{1}{60 + \text{rank}_{\text{channel}}(m)}$$

FTS scoring. FTS rows arrive from the full-text channel without a vector distance. Rather than displaying a synthetic 1.00 (which made irrelevant keyword hits appear as perfect matches), each FTS-only row is annotated with its *true* cosine distance against the query vector; a row whose stored vector is missing or zero-length is assigned the worst-case distance (similarity -1) instead.

Temporal decay is not a second retrieval channel — it is an additional weighting term applied only to agent-scoped memories (non-empty `agent_handle`). An `agent_handle` filter therefore narrows the result set and enables decay; it does not switch retrieval modes.

$$w = e^{-\lambda \cdot h}$$

where λ is the decay rate and h is hours since ingestion. Unscoped documents (`agent_handle` = "") are exempt from decay (production leaves $\lambda = 0$).

8.5 Retrieval Seam and Evaluation

The `src/quarry/retrieval/` package is a deliberate seam (DES-037): the same `HybridRetriever` that serves production also serves the retrieval evaluation harness (`make eval`), so the committed baseline measures the retriever that actually ships. `RetrievalConfig` exposes the levers a bake-off varies — RRF k, over-fetch multiplier, distance metric, exact-vs-ANN vector search, reranker, and embedding strategy — without forking the code path. The harness is internal dev/CI tooling; it is not an end-user command. See `docs/eval-harness-design.md`.

Three columns extend the chunks table for agent-scoped memory:

Column	Type	Default	Purpose
`agent_handle`	utf8	""	Agent identity scope
`memory_type`	utf8	""	fact, observation, opinion, procedure, lesson
`summary`	utf8	""	One-line summary

Migration is idempotent — `_migrate_schema()` adds missing columns with empty-string defaults on every table open. Existing databases gain the columns transparently.

`memory_type` is a closed vocabulary (`src/quarry/memory_types.py`, `MemoryType`): the four agent-owned types decay; `lesson` is written only by `learn` and is boosted instead of decayed. Every write route — `/v1/remember`, `/v1/ingest`, `/v1/capture` — validates through one `RouteGroup` method, so an unknown value is a 400 with an identical body on all three, never a silently stored row that neither decays nor matches a typed filter (DES-055).

Identity tagging: the parent-session hooks (PreCompact, SessionEnd) read the agent handle from the ethos repo pin — `.punt-labs/ethos.yaml` first, then the legacy `.punt-labs/ethos/config.yaml`, at

each ancestor up to the repository boundary (`src/quarry/ethos_tree.py`, `src/quarry/ethos_handle.py`) — and tag captured transcripts with it. SubagentStop never uses the pin for attribution (the pin names the leader): it uses `agent_type` iff that handle is a registered identity in the vendored or global ethos tree, else the capture is filed unattributed. A `remember` carries the caller's own handle; the daemon never infers one from `cwd` (DES-055).

8.6 Ethos Extension Integration

Quarry stores its configuration in ethos's generic extension mechanism (DES-008, DES-019):

```
~/.punt-labs/ethos/identities/<handle>.ext/quarry.yaml   (global)
.punt-labs/ethos/identities/<handle>.ext/quarry.yaml    (vendored, per repo)
```

The extension file contains two categories of data:

1. **Sidecar config** — `memory_collection` and `expertise_collections` are read directly by quarry via the filesystem (no ethos API call).
2. **Session context** — `session_context` is a YAML literal block scalar emitted verbatim by ethos at session start and before context compaction. It contains the agent's memory instructions (collection name, slash commands, memory types).

The `session_context` carries a versioned `## Memory (quarry guide v2)` block (`src/quarry/ethos_ext_block.py`): the five moments to `remember`, what never to store, and the rule that an agent always passes its own handle. `quarry install` refreshes the global identities; `quarry enable` refreshes the vendored `.punt-labs/ethos/identities/<handle>.ext/quarry.yaml` files (a committed diff, reported as "commit via PR"); `doctor` stays read-only. A v1 block is replaced in place, a customised block is left alone, and per-identity outcomes (`src/quarry/ethos_ext_scan.py`) ensure one malformed file does not abort the rest.

Ownership boundary: ethos emits quarry's instructions without understanding them. Quarry writes its own instructions without importing ethos. The dependency is one-way — quarry depends on ethos for identity; ethos has zero knowledge of quarry's internals.

8.7 Agent-Memory Write Loop (DES-055)

Recall-on-launch had worked since the agent-memory schema landed: ethos injects each identity's `session_context` into a spawned agent. The *write* side stayed dormant — no `memory-*` collection existed — until DES-055 added the three producers that fill it. None of them runs a model; each is a thin client of the daemon's scrub-before-store `/v1/remember` route.

Loop 1. **Persist habit.** The memory guide above tells every agent when to `remember` and to pass its own handle. The daemon routes a handle-bearing `remember` with an empty `collection` to `memory-<handle>`.

Loop 2. **Evaluator feedback.** `quarry missions sync` (CLI), `missions_sync` (MCP), and `/quarry missions sync` (plugin) read the repo's `.punt-labs/ethos/missions/` trio (`contract`, `results`, `reflections`) and file each *frozen* round as an `observation` in `memory-<worker>`, named `mission-<repo>-<id>-r<n>`. Idempotent: a round the daemon already holds is skipped, a name held by another checkout's round is an error and never overwritten (`--force` re-files a matching key only), `--dry-run` posts nothing. Ethos is never called, so the one-way dependency holds.

Loop 3. **Capture distillation.** `SubagentStop` archives the subagent's own transcript to `<repo>-captures` and, when `agent_type` names a registered identity, files the subagent's final assistant turn — already a summary by construction — as an `observation` in `memory-<handle>` (`subagent-<id8>-report`). A bare `Agent()` (`general-purpose`) is filed unattributed.

Module	File
Vocabulary	src/quarry/memory_types.py
Ethos tree	src/quarry/ethos_tree.py, src/quarry/ethos_handle.py
Mission store	src/quarry/mission_store.py, src/quarry/mission_records.py, src/quarry/mission_round_p
Mission memory	src/quarry/mission_memory.py, src/quarry/mission_sync_types.py
Surfaces	src/quarry/cli_missions.py, src/quarry/mcp_missions.py
Subagent	src/quarry/subagent_capture.py, src/quarry/subagent_report.py

8.8 Recall Telemetry (DES-056)

DES-055 gave agents a place to write memory; DES-056 answers whether recall is actually exercised. src/quarry/daemon/routes/search.py times every SearchService.search() call and, when Settings.telemetry_ena scrubs the query text through the same Scrubber pass captures use (DES-041) and records one query_events row plus one query_hits row per ranked result to a local, engine-free SQLite store (src/quarry/query_log.py + src/quarry/query_log_schema.py), mirroring src/quarry/sync_registry.py's WAL/busy-timeout connection lifecycle. The write is boundary I/O: wrapped in try/except, a failure is logged and swallowed so a telemetry outage never turns a search into a 500.

GET /v1/insights mirrors the status chain across all three surfaces (quarry insights, the insights MCP tool, the HTTP route) and reports total queries, empty-result rate, p50/p95 latency, top empty (scrubbed) queries, per-collection and per-agent recall counts, a memory-vs-knowledge split, and a hit recency (decay-band) breakdown. The decay-band join — a hit's document identity against the live catalog's ingestion_timestamp — runs in the route, not the store: the query log has no LanceDB import, so only the code already holding ctx.database can perform it. SearchRequest carries an optional surface field (cli/mcp/http/ plugin) for provenance; the CLI find path stamps surface="cli" the same way the MCP find tool stamps "mcp".

8.9 Skills and Cross-Harness Distribution (quarry-x7ja)

Quarry ships two Agent Skills at plugin/skills/: quarry-recall (retrieve — understand code, recall a project decision, recall memory, all backed by quarry find with different scope flags) and quarry-capture (persist — remember/learn/ ingest, the five-moments timing, the agent_handle discipline). Both drive the quarry CLI directly rather than naming MCP tools, so the same skill content works unmodified on any harness; deep detail (the failure-mode recovery table, scoping and decay, the memory-type vocabulary) lives in each skill's references/ for progressive disclosure, while the SessionStart trigger trailer, the MCP server's instructions block, and the ethos-injected memory guide are kept to short pointer sentences naming the two skills, rather than each independently restating the same how-to.

Claude Code reads plugin/skills/ directly through the marketplace plugin tree — no deposit step. src/quarry/skills_install.py (SkillsInstaller) composes src/quarry/skills_source.py (SkillSource, which owns every source-tree and single-deposit mechanic: locate, hash, manifest, the atomic copy-and-swap) and adds only harness orchestration on top — which harness, which root. Together they copy the same two skill directories, verbatim, into the auto-load location each other harness

was confirmed (not guessed) to read skills from:

- **pi** — `~/.pi/agent/skills/<name>/`, matching `getAgentDir()`'s resolution to `~/.pi/agent`.
- **opencode** — `~/.config/opencode/skills/<name>/`, one of the paths opencode's own skill loader globs (`{skill,skills}/**/SKILL.md`) as its "global skills" root.
- **codex** — `~/.codex/skills/<name>/` (or `$CODEX_HOME`), the location codex's own preinstalled `skill-installer` skill deposits third-party skills into, explicitly alongside the sibling `.system/` directory that holds codex's own preinstalled skills — never inside it.

Each deposit is idempotent and version-stamped: a `.quarry-skill.json` manifest beside the deposited `SKILL.md` records a short content hash, so a re-run with unchanged source content is a no-op and a changed source overwrites the stale copy wholesale. The copy itself is swapped in atomically — written into a temp sibling directory first, the previous deposit (if any) renamed to a backup sibling, then the temp directory renamed into place, with the backup restored on any rename failure — so an interrupted write never leaves a half-deposited skill on disk (the file-I/O safety class this repo's review history flags first).

`quarry skills install [--agent <id>|--all]` and `quarry skills status` (`src/quarry/cli_skills.py`) expose this: `--all` (the default) targets every harness `SkillsInstaller.detected_harnesses()` finds on the machine (a harness's own top-level config directory, e.g. `~/.codex`, existing); `--agent` targets one harness explicitly, detected or not.

`quarry disable` retracts the deposits again through a private `_SkillRetraction` collaborator, gated by two guards, both required (defense-in-depth for an `rmtree` under the operator's real `$HOME`): **identity** — every marketplace-layout Claude Code plugin (lux, prfaq, dungeon, punt-kit, quarry itself) ships an identically-shaped `plugin/skills/` tree, so that shape alone proves nothing; only a checkout whose own `pyproject.toml` names the `punt-quarry` package is quarry's own, checked on the disabled repo's root directly, never via an ancestor walk (an ancestor walk would true-positive on every path nested inside the quarry checkout, including every test's own temp directory, and reach the operator's real home directory as a side effect of an unrelated `disable` call); and **manifest ownership** — `SkillsInstaller.remove`/`remove_all` refuse (`action="foreign"`) any same-named directory that lacks a `.quarry-skill.json` manifest, so a directory this installer never deposited is never an `rmtree` candidate no matter how it got there. Retraction is best-effort: a filesystem failure during either guard's checks or the removal itself is logged and swallowed, never aborting `disable` or blocking the marker/import teardown that already committed.

9 Logging and Exceptions

9.1 Logging Standard

Every module that performs I/O declares `logger = logging.getLogger(__name__)`. Pure data modules (`models.py`, `types.py`, `page_chunker.py`) do not.

Logging is configured once at each application entry point — the CLI, the MCP server, and the `quarryd` daemon (DES-048; the daemon writes its own `quarryd.log` with crash hooks for all three uncaught-exception routes). Library modules never call `logging.basicConfig`.

Level	Use
`DEBUG`	Internal state: variable values, branch decisions, intermediate results.
`INFO`	Operational milestones: starting a job, completing a step, resource counts.
`WARNING`	Unexpected but recoverable: empty input, deprecated parameter usage.
`ERROR`	Failures that prevent completion but do not crash. Always with exception context.

`CRITICAL` is not used. If the process must exit, it raises an exception.

Log messages use `%s`-style formatting (lazy evaluation), not f-strings. Each message includes enough context to trace an operation without reading code.

9.2 Exception Handling

The codebase uses built-in exception types. No custom exceptions unless a caller needs to distinguish failure modes programmatically.

Exception	Raised when
`ValueError`	Invalid input: unsupported format, bad parameter.
`FileNotFoundError`	A file path argument does not exist.
`RuntimeError`	An operation reports failure (OCR engine error, ingestion failure).
`TimeoutError`	An external service exceeds its time budget.

Rules:

1. Never swallow exceptions. Every `except` block must re-raise, or log with `logger.exception()` and raise a different exception, or (at system boundaries only) return a documented sentinel.
2. Never use bare `except:`.
3. Never catch `Exception` broadly in library code. Broad catches are permitted only at the outermost boundary (MCP tool handler, CLI command).
4. Exceptions carry context: include the values that caused the failure.
5. Use `try/finally` for cleanup, not `try/except`.
6. Do not use exceptions for flow control.

10 Security

10.1 Bearer Token Authentication

`quarryd --api-key` (or `QUARRY_API_KEY` env var) gates all HTTP endpoints except `/health` behind `Authorization: Bearer <key>`.

- `/health` is exempt — load balancers need unauthenticated health checks.
- `OPTIONS` is exempt — CORS preflight requests carry no auth.
- `hmac.compare_digest` for token comparison — prevents timing attacks.
- Case-insensitive scheme per RFC 7235.
- Empty key = no auth — prevents accidental trivial bypass.
- No key = auth disabled — local use requires no key.

The server refuses to bind to a non-loopback address without `--api-key`.

10.2 Request Admission (CSRF)

The daemon serves REST only, so there is no WebSocket to hijack; the cross-origin threat is a forged browser `POST`. Two checks compose the admission trust model:

- `JsonContentTypeGuard` — an ASGI choke point that rejects any `POST` without `Content-Type: application/json`. A browser cannot send that header cross-origin without a preflight the CORS policy refuses, so a forged cross-origin write is blocked at the door.
- The `serve.token` / API-key bearer (checked on every `/v1` request) blocks an unauthorized *local* reader.

10.3 CORS

`CORSMiddleware` configured with GET and OPTIONS methods, Authorization and Content-Type headers. Default allowed origin: `http://localhost`. Configurable via `--cors-origin` (repeatable).

The server reflects the request's `Origin` header only when it matches the allow list, and adds `Vary: Origin` to signal cacheability varies by origin. When no `Origin` is present, no CORS headers are emitted.

10.4 TLS and TOFU Certificate Pinning

The installed service always runs with TLS enabled. `quarryd` without `--tls` is supported for local development but must not be used for production. Certificate management uses TOFU (Trust On First Use) with self-signed CA pinning (DES-020).

Certificate generation: `quarry install` generates an EC P-256 CA and server certificate. Files are written atomically to `~/.punt-labs/quarry/tls/` with restricted permissions (0600 for keys, 0644 for CA cert). The server cert SAN includes the configured hostname (via `QUARRY_TLS_HOSTNAME`), localhost, and loopback addresses.

TOFU login:

1. Client fetches `/ca.crt` over HTTPS (verification disabled — bootstrap).
2. Displays SHA256 fingerprint for out-of-band confirmation.
3. Pins CA cert at `~/.punt-labs/mcp-proxy/quarry-ca.crt`.
4. All subsequent connections verify against the pinned CA only — system roots excluded.

Pinned-CA consumers: `QuarryClient` builds its TLS context from the pinned CA as the sole root (system roots excluded), so every daemon RPC — CLI, `quarry mcp`, hooks, library — verifies against it. The `quarry.toml` profile also records the CA under `ca_cert`, which `mcp-proxy` reads (as the sole root, TLS 1.3 minimum) when it transports MCP for other MCP clients; it is no longer on quarry's own MCP path (DES-031 v2.2).

10.5 Log Safety (CWE-532, CWE-117)

Uvicorn's access log is disabled (`access_log=False`), eliminating query string leakage at the source. The search handler logs only result count, never the raw query. (The WebSocket session-key sanitization the prior design described was removed with the in-daemon `/mcp` route — there is no longer a WebSocket surface to log.)

10.6 Capture PII Redaction (DES-036)

Captures — session transcripts and WebFetch auto-captures — are the input to the planned private shadow-repo sync (quarry-ow3k), so un-redacted content in a capture is a leak into a git-tracked (and soon pushed) surface. Every capture is scrubbed **at write time**, fail-closed, before any bytes reach disk.

Scrubber (`src/quarry/scrub.py`). A `Scrubber` applies redaction passes in a load-bearing order: *secrets* → *paths* → *emails* → *hostname* → *profanity*. Email must precede hostname so a hostname inside an email domain is subsumed by the whole-email redaction rather than half-redacted (which would leak the local part). Each pass emits a marker no pass can re-match, so scrubbing is idempotent — backfill may safely re-run.

- **Secrets** (`src/quarry/scrub_rules.py`): GitHub PATs, AWS keys, Anthropic/OpenAI keys, bearer headers, JWTs, PEM/GPG private key blocks, `KEY=value` env secrets, Slack tokens.
- **Paths**: home directories (`/Users/<user>/`, `/home/<user>/`) collapse to `~`, retaining the useful deeper structure.
- **Emails**: any RFC-shaped address → `[REDACTED:email]`. A placeholder redacts every address (completeness beats attribution), not an ethos-handle mapping.
- **Hostname**: bounded to the local machine name (`socket.gethostname()` + `.local` + short leaf), not arbitrary dotted-token detection — the latter has a catastrophic false-positive rate on identifiers like `github.com` or `quarry.db.facade`.

Single write choke point (src/quarry/capture.py). Every local .md capture producer — SessionTranscriptCapture (src/quarry/session_transcript.py, serving the PreCompact, SessionEnd, and SubagentStop hooks) and the backfill path (src/quarry/backfill.py) — routes through one CaptureWriter. Scrubbing runs to completion before an atomic temp-file rename, so a scrub failure writes no file and a partial or half-redacted capture is never left behind.

WebFetch DB-ingest. The WebFetch auto-capture path scrubs page text via an opt-in content_scrubber on the UrlIngest/InlineIngest request (src/quarry/ingestion/web_ingest.py), which the daemon's CaptureIngestJob and ScrubbedIngestJob (src/quarry/daemon/ingest_jobs.py) set for every hook-originated capture. The field defaults to None: user-initiated quarry ingest <url>, sitemap crawls, and directory sync are *not* scrubbed — redaction is a captures concern, not a general-ingestion one. The URL's persisted metadata form is its admitted HttpUrl.document_name (src/quarry/http_url.py), scheme://host[:port]/path with no userinfo, query, fragment, or ;params, run through the same text scrubber (DES-057).

11 Deployment

11.1 Single Machine

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
```

Installs quarry, embedding model, mcp-proxy, Claude Code plugin, and registers a daemon with TLS on localhost.

11.2 Network Mode (Remote GPU)

```
export QUARRY_API_KEY=$(openssl rand -hex 32)
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh -s -- --network
```

Same as default, but binds daemon to 0.0.0.0:8420 instead of localhost. Requires QUARRY_API_KEY. Installs quarry, embedding model, generates TLS certificates, registers systemd service. Detects NVIDIA GPUs and swaps onnxruntime for onnxruntime-gpu. Prints CA fingerprint for client TOFU. Claude Code plugin installed if claude CLI is on PATH.

11.3 Client (Remote Access)

```
curl -fsSL https://raw.githubusercontent.com/.../install.sh | sh
quarry login <server> --api-key <token>
```

No special flag needed. Default install runs a local daemon on localhost. quarry login redirects queries to the remote server via TOFU certificate pinning and writes the client profile (~/.punt-labs/mcp-proxy/quarry.
URL, pinned CA, bearer) that ClientConfig resolves for every surface.

11.4 Daemon (quarryd)

```
QUARRY_API_KEY=<token> quarryd --port 8420 --host 0.0.0.0 --tls
```

Non-loopback hosts require an API key (the server refuses to start without one).

Managed by launchd (macOS) or systemd (Linux) via quarry install. API key delivered via systemd EnvironmentFile (Linux) or launchd EnvironmentVariables (macOS) — never in process arguments.

11.5 Container (Fly.io)

Multi-stage Dockerfile, three stages:

1. **deps** (python:3.13-slim): installs uv, runs uv sync --frozen --no-dev. Two-pass for Docker layer caching (deps first, then source).

2. **model** (extends `deps`): downloads the embedding model at build time (∼120 MB, int8 quantized). Cached as a separate Docker layer.

3. **runtime** (`python:3.13-slim`): copies `/app` from deps, copies model cache from model stage. `QUARRY_ROOT=/data`, port 8080.

LanceDB data lives on a Fly persistent volume at `/data`. `QUARRY_API_KEY` is set via `fly secrets`. Cold start is ∼5 s with the baked-in model.

12 Performance

12.1 Critical Path

Operation	Latency	Notes
Model load	∼250 ms	One-time per process
Embedding (1 chunk)	∼50 ms	Apple Silicon, CPU
LanceDB search	<10 ms	Vector similarity + filter
End-to-end search	<100 ms	After model is loaded

12.2 Fire-and-Forget Writes

Side-effect operations return immediately with a `task_id`; the work runs on the daemon's serialized per-`RouteKey` queue (DES-042). This is necessary because:

1. MCP tool calls block the LLM's response stream. A 30-second ingest would block Claude from responding. The 202 unblocks the caller immediately.

2. Bounded, non-blocking admission (503 on a full queue) prevents the resource exhaustion that unbounded `asyncio.create_task` ingests caused (the load-90 starvation, quarry-lnog).

3. One in-flight `progressive_insert` per LanceDB table (single writer per `RouteKey`) plus the embed `Semaphore` clamped to 1 eliminate the concurrent delete-then-add lost-update race, without snapshotting mutable per-request state.

12.3 Single Table Design

All chunks live in one LanceDB table (`chunks`). Document and collection boundaries are columns, not separate tables. This simplifies cross-document search and avoids table proliferation. Filtering by document/collection/page_type/source_format uses LanceDB's built-in filter predicates on the vector search.

13 Plugin and Hook System

The Claude Code plugin (`plugin/hooks/hooks.json`, each script a one-line `quarry-hook <event>` dispatched by `src/quarry/_hook_entry.py`) registers hooks that fire on session events and tool calls. All hooks are fail-open: exceptions are logged and the process exits 0. Every hook is a thin daemon client — no hook imports the engine — and every capture lands in the `<repo>-captures` collection the daemon derives from the hook's `cwd` (`default-captures` when the directory is unregistered), scrubbed server-side before storage. Each is independently toggleable under `auto_capture` in `.punt-labs/quarry/config.md`; only `read` defaults to off.

Hook Event	Matcher	Effect	Blocks?
SessionStart	*	Auto-register directory + background sync	No
PostToolUse	quarry tools	Suppress tool output formatting	No
PostToolUse	WebFetch	Auto-ingest fetched URL (scrubbed)	No
PostToolUse	WebSearch	File a scrubbed digest of the results	No
PostToolUse	Read	Opt-in: capture an out-of-tree prose file	No
PreCompact	*	Capture transcript before compaction (scrubbed)	No
SessionEnd	*	Capture the full transcript on every close	No
SubagentStop	*	Archive subagent transcript; distill its report into `memory-<handle>`	No[†]

[†]Claude Code treats `SubagentStop` as a blocking hook: a non-zero exit or a `decision` field keeps the subagent running. The handler returns {} on every path, and its tests assert that under adversarial payloads.

13.1 Session Start

`handle_session_start()` auto-registers the working directory and launches a background sync via a detached subprocess. Single-flight is a kernel `flock` on `~/.punt-labs/quarry/sync.pid` (`SyncLock`, `src/quarry/sync_lock.py`) held for the sync subprocess's lifetime through fd inheritance, so a dead holder's lock is released by the kernel with no PID parsing or reclaim race. Returns `additionalContext` immediately without waiting.

13.2 Web Fetch Capture

`handle_post_web_fetch()` hands the already-fetched page to the daemon's `/v1/capture` route, which extracts, scrubs, and stores it in `<repo>-captures` (no second fetch; a JS-rendered page that extracts to zero chunks is re-fetched server-side through the SSRF gate). Before sending, `WebFetchLoopCloser` (`src/quarry/web_fetch_loop_closer.py`) checks `/v1/captures/lookup` for a prior capture of the URL and, when one exists, nudges the agent to `find` it instead of re-fetching. The URL's persisted metadata form is derived by `WebFetchName` from the admitted `HttpUrl` (DES-036, DES-057).

13.3 Web Search and Read Capture

`HookAgent.post_web_search()` files a scrubbed digest of a WebSearch's results under the document name `search: <query>`. `HookAgent.post_read()` is opt-in (`read: false` by default, because `Read` fires on every file the agent opens): a file passes four admission checks in order — outside every registered tree, not on the secret-path denylist, an allow-listed extension, under the size cap — plus a client-side secret-pattern scan, before its text is sent.

13.4 Pre-Compact and Session-End Capture

`handle_pre_compact()` and `HookAgent.session_end()` run the same `SessionTranscriptCapture` pipeline (`src/quarry/session_transcript.py`): archive the raw JSONL under `~/.punt-labs/quarry/sessions/`, extract user/assistant text (tool-use blocks skipped, capped at 500,000 characters), write the scrubbed `.md` next to the project through the shared `CaptureWriter`, and POST a `CaptureIngestRequest` the daemon files as `session-<id8>` in `<repo>-captures`, attributed to the repo's ethos pin. PreCompact fires only on compaction; SessionEnd fires on every close, so a short session that never compacts still yields a durable capture. A down daemon leaves the local archive for `quarry backfill-sessions`.

13.5 Subagent Stop Capture

`HookAgent.subagent_stop()` captures the subagent's own transcript (`agent_transcript_path`, keyed by `agent_id` — the parent's `transcript_path`/`session_id` are not its), then — only when `agent_type` names a registered ethos identity and the transcript had text — files the subagent's final assistant turn as an `observation` in `memory-<handle>` named `subagent-<id8>-report`, through `/v1/remember` with the same 5 s cap as the first post (`src/quarry/subagent_capture.py`, `src/quarry/subagent_report.py`). The transcript is parsed once for both rows. No model distills anything: the report is the summary the subagent already wrote. A non-identity `agent_type` (`general-purpose`) gets the raw capture only, never a row under the leader's handle (DES-055).

14 CLI Commands

Command	Purpose
`ingest <url>`	Ingest a URL (URL-only; local files and directories go through `register` + `sync`). Flags: `--agent-handle`, `--memory-type`, `--summary`
`remember`	Ingest stdin content (`--name`; `--agent-handle` routes to `memory-<handle>`; `--memory-type` from the closed vocabulary)
`learn <lesson>`	Save a distilled lesson (`--topic`, `--name`); the only writer of `memory_type = lesson` (DES-053)
`find <query>`	Semantic search
`show <document>`	Document metadata or page text
`status`	Database statistics
`use <name>`	Set persistent default database
`delete <name>`	Delete document or collection
`register <directory>`	Register for incremental sync
`deregister <collection>`	Remove directory registration
`sync`	Sync all registered directories
`enable [directory]`	Enable quarry knowledge capture for a project directory
`disable [directory]`	Disable quarry knowledge capture; remove registration and data
`optimize`	Compact LanceDB table and rebuild indexes (CLI-only). Flags: `--force`
`list documents`	List indexed documents
`list collections`	List collections with counts
`list databases`	List named databases
`list registrations`	List registered directories
`mcp`	Start MCP server (stdio; the engine runs only in `quarryd`)
`missions sync`	File each frozen ethos mission round into `memory-<worker>` (`--mission`, `--dry-run`, `--force`; DES-055)
`captures init|push`	Bootstrap / re-scrub and push the project's private capture shadow repo
`backfill-sessions`	Backfill historical Claude Code session transcripts (returns 202)
`login <host>`	TOFU login to remote server (cert pinning)
`logout`	Disconnect from remote server
`remote list`	Show remote config; `--ping` validates connection
`install`	Set up data directory, download model, generate TLS certs, register daemon, detect GPU, configure ethos ext
`doctor`	Check environment health
`uninstall`	Remove system daemon

Command	Purpose
`version`	Print version

Global flags: `--json`, `--verbose`/`-v`, `--quiet`/`-q`, `--db <name>`.

Remote routing (DES-021): When a remote server is configured (via `quarry login`), all data commands should route to the remote server over HTTPS with the pinned CA cert. Only authentication commands (`login`, `logout`, `remote`) and administration commands (`install`, `uninstall`, `mcp`, `version`) are local-only. All data commands — `find`, `show`, `status`, `ingest`, `remember`, `learn`, `delete`, `sync`, every `list` subcommand, directory `register`/`deregister`, and `missions sync` (which reads the local mission files and posts each round to the configured daemon) — route remotely. `use` is local-only by design (the remote server is fixed to its own database).

15 Test Architecture

The default run (`make test`) deselects the `slow` and `embedding` tiers; `uv run pytest --collect-only -q` reports the current test count. Three tiers plus integration; the files named are representative, not exhaustive:

Tier	Strategy	Files
Tier 1: Core logic	Real data, no mocks	test_database, test_page_chunker, test_embeddings, test_code_processor, test_html_processor, test_text_processor, test_collections, test_config, test_formatting, test_models, test_retrieval_fusion, test_retrieval_hybrid, test_memory_types, test_scrub, test_format_strategies, test_registry
Tier 2: Orchestration	Mocked I/O	test_pipeline, test_pipeline_images, test_ocr_local, test_url_ingestion, test_sitemap, test_backends, test_sync, test_sync_concurrency, test_hooks, test_hooks_agent, test_subagent_capture, test_mission_memory, test_mission_store, test_doctor
Tier 3: Surface wiring	Fully mocked / hermetic in-process ASGI (httpx.ASGITransport), <2 s	test_cli, test_cli_missions, test_mcp_server, test_mcp_missions, test_http_server (the src/quarry/api/ + src/quarry/daemon/routes/ contract), test_api, test_client, test_client_config, test_retrieval_equivalence, test_daemon_launcher, test_watch_loop, test_watch_reconcile, test_fs_watchdog, test_proxy, test_service
Invariants	Long-lived-process leak guards; suite hermeticity (DES-047)	test_resource_invariants, test_hermeticity
Integration	Real ONNX + real LanceDB; real loopback TLS	test_integration, test_tls_smoke (@pytest.mark.slow; the model loads only under embedding)

Quality gates (make check): ruff (lint + format), mypy (strict), pyright (strict), pytest (all markers except slow/embedding), the OO, coupling, and suppression merge-base ratchets (DES-040), the import-linter client/engine contract, and the OpenAPI drift check. Zero violations required.

16 Formal Model

The plugin state machine — how quarry hooks into the Claude Code session lifecycle — is formally specified in `docs/claude-code-quarry.tex` using Z notation (ISO 13568).

The specification covers:

- Session start with/without auto-sync
- Web fetch auto-capture into the `<repo>-captures` collection
- Pre-compaction transcript capture into `<repo>-captures`
- Database switching via `use` tool
- Knowledge hint injection at tool boundaries (specified, not yet built)
- Compaction observability: a capture flag records whether capture preceded compaction; the hook system enforces capture-before-compaction on a best-effort basis
- Clean session boundaries: all quarry state cleared on `EndSession` — now wired to the `SessionEnd` hook, which also captures the full transcript
- The WebSearch, Read, and SubagentStop captures ride base-layer operations (`CompleteToolSuccess`, `StopSubagent`); the spec's implementation-validation section records them as wired rather than modelling them as quarry-layer schemas

The specification can be type-checked with `fuzz -t claude-code-quarry.tex`.

A Operation Concurrency Model

Every quarry operation is exposed on up to three surfaces: the HTTP API (`/v1`, daemon), the CLI, and the MCP server (`quarry mcp` stdio). The concurrency model differs by surface because each has different constraints: HTTP connections time out, MCP tool calls block the LLM response stream, and CLI commands run in a user's terminal. Under daemon-first every CLI and MCP call is a `QuarryClient` request to the daemon, so the "local" and "remote" CLI columns below differ only in transport target, not in code path.

Table 4 classifies each operation. The **Client model** column describes what the caller experiences:

sync The caller blocks until the operation completes and receives the result inline.

fire-and-forget The caller receives an acknowledgement immediately (HTTP 202 or a short string). Work runs in the background; the caller polls or ignores the outcome.

— The operation is not available on that surface.

A.1 Concurrency Matrix

Table 4: Operation concurrency by surface. *Long-running* = involves network I/O, embedding, or bulk LanceDB writes.

Operation	HTTP API	CLI (local)	CLI (remote)	MCP
Read operations				
find (search)	sync	sync	sync	sync
show	sync	sync	sync	sync
status	sync	sync	sync	sync
list documents	sync	sync	sync	sync
list collections	sync	sync	sync	sync
list registrations	sync	sync	sync	sync

Operation	HTTP API	CLI (local)	CLI (remote)	MCP
list databases	sync	sync	sync	sync
health	sync	—	—	—
Mutating operations				
register	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
deregister	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
delete (document)	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
delete (collection)	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
remember	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
learn	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
ingest (URL)	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
capture (hooks)	fire-and-forget	—	—	—
sync	fire-and-forget	fire-and-forget	fire-and-forget	fire-and-forget
missions sync	— (N × remember)	sync	sync	sync
use (switch db)	sync	sync	sync	sync
optimize	fire-and-forget	fire-and-forget	fire-and-forget	—
backfill-sessions	fire-and-forget	fire-and-forget	fire-and-forget	—

A.2 Design Rationale

Read operations are synchronous on all surfaces. Hybrid search (embedding + vector similarity + BM25) completes in <100 ms after model load; the overhead of task tracking would exceed the operation itself.

Mutating operations are all fire-and-forget on HTTP, and therefore on every client of it: the CLI prints the daemon's `task_id` and exits 0. Every mutating endpoint returns 202 Accepted with a `task_id`; the work runs on the daemon's serialized per-`RouteKey` ingest queue (DES-042). This eliminates the 15-second CLI timeout as a failure mode for any write operation. Local files never travel over the wire: `ingest` is URL-only on every surface, and a directory is indexed by `register` + the daemon's own watch loop. `missions sync` is the one client-side composite: it reads the mission files locally and issues one `remember` per frozen round, so it has no HTTP row of its own.

- All mutating endpoints use the single unified `GET /v1/tasks/{task_id}` for status polling; the old `/sync/{id}` and `/ingest/{id}` alias routes were removed in the DES-031 v2.2 contract rewrite.
- **Sync** no longer returns 409: it enqueues transparently (202 + poll), because the daemon's serialized per-`RouteKey` queue (DES-042) already prevents concurrent writers to a table. The 409 is retained only for **optimize** and **backfill-sessions**, whose whole-table compaction/rebuild must not overlap itself.
- Completed and failed tasks are garbage-collected after a 1-hour TTL on the next task creation.
- **Optimize** and **backfill-sessions** are exposed on HTTP and the CLI but *not* as MCP tools — they are whole-database maintenance, run with explicit intent rather than from an agent's tool loop.

A.3 Task Lifecycle

All background tasks follow the same lifecycle:

1. `POST` validates input (including the closed `memory_type` vocabulary on the write routes), creates a `TaskState` record in the daemon's `TaskRegistry` (`src/quarry/daemon/tasks.py`), hands the job to the daemon's background machinery (the per-`RouteKey` ingest queue for writes), and returns HTTP 202 with `task_id` and `status` = `accepted`.
2. The background task updates `status` to `completed` (with `results`) or `failed` (with `error`) on completion.
3. A `finally` guard marks any task that exits without setting a terminal status as `failed`.

4. `CancelledError` is caught before `Exception` to ensure clean shutdown on server stop.
5. The CLI prints the `task_id` and exits 0 (fire-and-forget).